

Tehnički vodič — detaljna verzija

Detekcija klikbejt naslova u sportskim vestima na srpskom jeziku

Obrada prirodnih jezika 2025/2026 — Elektrotehnički fakultet

Ovo je **detaljna verzija** tehničkog vodiča: prolazi kroz *ceo izvorni kod* projekta, fajl po fajl, ali sa *proširenom teorijom* — svaki pojam je objašnjen do nivoa na kome treba da bude jasan i bez prethodnog znanja, uz konkretne primere i brojke. Sažeta verzija je `Tehnicki-vodic-kroz-kod.pdf`; pregled „gde se šta nalazi” u `docs/Mapa-koda.md`, a komande za pokretanje u `docs/Komande.md`.

Sadržaj

1	Arhitektura i tok podataka	1
2	Faza 1 — Prikupljanje podataka (<code>src/scraping/</code>)	2
2.1	<code>sportklub_api.py</code> — glavni skrapper	2
2.2	<code>common.py</code> , <code>merge.py</code> i ostali	2
3	Faza 2 — Anotacija (<code>src/annotation/</code>)	3
3.1	<code>make_splits.py</code> — podela posla	3
3.2	<code>iaa.py</code> — Cohen’s kappa	3
3.3	<code>build_dataset.py</code> — balansiranje	4
3.4	<code>reconcile_iaa.py</code> , <code>stats.py</code>	4
4	Pretprocesiranje teksta (<code>src/preprocessing/serbian.py</code>)	4
5	Faza 3a — Osnovni modeli (<code>src/baseline/</code>)	5
5.1	<code>run_baseline.py</code> — grid eksperimenata	5
5.2	<code>common.py</code> — učitavanje i metrike	7
6	Faza 3b — Enkoderski LLM (<code>src/transformers/</code>)	7
6.1	<code>finetune.py</code> — fine-tuning BERTić i mBERT	7
7	Faza 3c — Dekoderski LLM (<code>src/decoder/</code>)	8
7.1	<code>prompts.py</code> — konstrukcija upita	8
7.2	<code>claude_eval.py</code> — evaluacija nad celim skupom	9
8	Faza 4 — Izveštaj (<code>src/report/</code> + <code>report/</code>)	9
8.1	<code>make_tables.py</code> — CSV → LaTeX tabele	9
8.2	<code>make_figures.py</code> — grafici	9
8.3	<code>izvestaj.tex</code> — glavni dokument	9
9	Kako sve teče zajedno (reprodukcija)	9

1 Arhitektura i tok podataka

Projekat je organizovan oko **četiri faze**, gde izlaz jedne faze ulazi u sledeću. Sav kod je u Python-u, podeljen u pakete pod `src/` po fazama.

Faza 1	scraping	==>	data/raw/*.csv	(sirovi naslovi)
Faza 2	anotacija	==>	data/annotated/dataset.tsv	(1100/1100) + IAA (kappa)
Faza 3	modeli	==>	results/*.csv	(baseline / enkoder / dekode)
Faza 4	izveštaj	==>	report/tables/*.tex + figures/*.png	==> izveštaj.pdf

Ključna ideja koja se ponavlja kroz kod: *determinizam i reproduktivnost* (fiksiran `seed=42` svuda gde se uzorkuje/meša) i *razdvojene zavisnosti po fazi* (`requirements/requirements-*.txt`), da bi se svaka faza mogla pokrenuti nezavisno.

2 Faza 1 — Prikupljanje podataka (src/scraping/)

2.1 sportklub_api.py — glavni skrapper

Prikuplja naslove sa portala SportKlub preko njegovog **WordPress REST API-ja**. Umesto „grebanja” HTML stranica, koristi se zvanični JSON endpoint `/wp-json/wp/v2/posts`, koji vraća čist naslov, datum i kategorije — mnogo pouzdanije od parsiranja HTML-a.

Kako radi (paginacija): u petlji se traži stranica po stranica (po 100 naslova), dok se ne sakupi traženi broj. Između zahteva stoji pauza (`time.sleep`) da se API ne preoptereći:

```
API = "https://sportklub.n1info.rs/wp-json/wp/v2"
while len(rows) < broj:
    params = {"per_page": 100, "page": page,
              "_fields": "title,link,date,categories"}
    r = requests.get(f"{API}/posts", headers=HEADERS, params=params, timeout=20)
    ...
    page += 1
    time.sleep(pause)          # ljubazno prema serveru
```

Čišćenje naslova. WordPress vraća naslove sa HTML entitetima (npr. `“`; umesto navodnika). Funkcija `_clean_html` ih dekodira i uklanja zaostale tagove:

```
def _clean_html(s):
    s = html.unescape(s)          # &#8220; -> "
    s = re.sub(r"<[^>]+>", "", s) # ukloni <...> tagove
    return s.strip()
```

Metapodaci. Svaki naslov se pakuje u `Headline` strukturu sa poljima: naslov, izvor, URL, datum objave, rubrika (ime kategorije, dobijeno iz `fetch_categories`) i datum prikupljanja — čime je ispunjen zahtev da se zabeleži jedinstveni izvor/URL.

Izlaz: `data/raw/sportklub_api_<datum>.csv`. Namerno se povlači *višak* (3000–4000) da bi se posle anotacije skup mogao izbalansirati.

2.2 common.py, merge.py i ostali

- **common.py** — zajednički alati skrapera: `Headline` struktura, `HTTP HEADERS`, `save_csv`, konstanta putanje `DATA_RAW`.
- **merge.py** — spaja više povlačenja i radi **deduplikaciju**: egzaktan (identičan tekst) i *približno* (near-duplicate, biblioteka `rapidfuzz`) da bi uklonio gotovo iste naslove.
- **rss_scraper.py** — rezervni izvor preko RSS-a.
- **mozzart_bulk.py** — skrapper za Mozzart; *napušten* (rad samo sa SportKlub-om).

3 Faza 2 — Anotacija (src/annotation/)

Cilj faze: od sirovih naslova napraviti **balansiran, označen** skup i izmeriti koliko se dva anotatora slažu (**Cohen's kappa**).

3.1 make_splits.py — podela posla

Deli skup *tačno na pola* (Filip / Danilo, deterministički `seed=42`). Da bi se mogla izračunati saglasnost, izdvaja **kalibracioni skup**: prvih 110 naslova svake polovine anotiraju *oba* člana (unakrsno), pa za 220 naslova postoje dve nezavisne oznake.

```
rnd.shuffle(rows)                # deterministički (seed 42)
A, B = rows[:pola], rows[pola:] # Filipova / Danilova polovina
cal_A, cal_B = A[:110], B[:110]  # kalibracioni (prvih 110 svake) -> anotira i drugi
```

Izlaz: data/annotated/{filip,danilo}_glavna.csv (sa praznom kolonom labela) i data/calibration/* (kalibracioni za drugog člana).

3.2 iaa.py — Cohen's kappa

Računa međuanotatorsku saglasnost. **Cohen's kappa** meri slaganje *iznad* slučajnog: ako se dva anotatora slažu samo zato što obojica često biraju istu klasu, kappa to „kažnjava”. Formula:

$$\kappa = \frac{P_o - P_e}{1 - P_e},$$

gde je P_o stvarni udeo slaganja, a P_e *očekivano* slaganje slučajem. Implementacija je svega nekoliko linija:

```
def cohen_kappa(a, b):
    n = len(a)
    po = sum(1 for x, y in zip(a, b) if x == y) / n          # stvarno slaganje
    pe = sum((a.count(c)/n) * (b.count(c)/n)                  # očekivano slučajem
              for c in set(a) | set(b))
    return (po - pe) / (1 - pe), po
```

Primer (naši brojevi). Od 220 kalibracionih, slažu se na 182 $\Rightarrow P_o = 0,827$. Filip je 93 naslova označio kao klikbejt (127 kao regularan), Danilo 81 (139 regularan). Očekivano slaganje:

$$P_e = \frac{127}{220} \cdot \frac{139}{220} + \frac{93}{220} \cdot \frac{81}{220} = 0,520.$$

$$\kappa = \frac{0,827 - 0,520}{1 - 0,520} = \mathbf{0,640}.$$

Skripta ispiše i interpretaciju po Landis–Kohovoj skali (0,61–0,80 = „dobro”) i listu svih neslaganja — ulaz za analizu spornih slučajeva u izveštaju.

Teorija — zašto kappa, a ne prosto slaganje

Prosto procentualno slaganje (P_o) procenjuje kvalitet kada je zadatak *neuravnotežen* ili anotatori imaju naviku da biraju istu klasu. Ako bi npr. oba anotatora nasumično pogađala, ali 80% naslova nazivala regularnim, slučajno bi se složili u $\approx 0,68$ slučajeva — a to nije nikakvo „znanje”. **Cohen's kappa** oduzima to očekivano slaganje slučajem (P_e) i normira ostatak: $\kappa = 1$ je savršeno slaganje, $\kappa = 0$ je „kao slučaj”, $\kappa < 0$ gore od slučaja. Zato je kappa standard za međuanotatorsku saglasnost: meri slaganje *koje se ne može objasniti pukom raspodelom klasa*. Za binarne oznake u ozbiljnim primenama poželjno je $\kappa \geq 0,8$; naših 0,640 je „dobro” i posledica suštinske subjektivnosti pojma klikbejt (detaljno u izveštaju).

3.3 build_dataset.py — balansiranje

Spaja dve označene polovine i pravi **balansiran** finalni skup: deterministički izmeša i uzme do 1100 naslova svake klase (1100 klikbejt / 1100 regularan):

```
rnd = random.Random(42)
rnd.shuffle(sel_kb); rnd.shuffle(sel_ne)    # po klasi
bal = sel_kb[:1100] + sel_ne[:1100]
rnd.shuffle(bal)                          # promešaj klase
```

Izlaz: data/annotated/dataset.tsv (format naslov\t labela — ulaz za Fazu 3) i nebalansirani dataset_full.csv (zadržava id/podtip). Balansiranje uklanja pristrasnost klasifikatora ka većinskoj klasi i pojednostavljuje tumačenje metrika.

3.4 reconcile_iaa.py, stats.py

- **reconcile_iaa.py** — pomaže pri *adjudikaciji* (usaglašavanju) spornih naslova posle IAA.
- **stats.py** — deskriptivna statistika finalnog skupa (raspodela klasa, prosečne dužine) → results/faza2_statistika.txt.

4 Pretprocesiranje teksta (src/preprocessing/serbian.py)

Centralni modul za **tokenizaciju, stemovanje i lematizaciju**; koristi ga Faza 3a. Detaljno objašnjenje tehnika sa primerima je u zasebnom dokumentu docs/Obrada-teksta.md; ovde sažeto i tehnički.

Lanac: basic_normalize (NFC + ćirilica→latinica + lowercase) → tokenize → opciono stem/lemma.

Tokenizacija regularnim izrazom — token je niz slova (sa srpskim dijakriticima) i cifara; interpunkcija je razdvajač:

```
# token = cifre + latinica + srpski dijakritici (zćčđš) + ćirilica (opseg A-S)
_TOKEN_RE = re.compile(r"[0-9a-zćčđš<ćirilica>]+")
"Partizan pobedio 2:1!" -> ["partizan", "pobedio", "2", "1"]
```

Stemovanje — heuristički, bez zavisnosti: za token se pohlepno (najduži prvo) uklanja najduži poznati srpski nastavak, uz zaštitu da osnova ne padne ispod 3 slova:

```
for suf in _SUFFIXES:                # sortirani od najdužeg
    if token.endswith(suf) and len(token)-len(suf) >= 3:
        return token[:-len(suf)]
"pobedom", "pobede", "pobedi" -> "pobed"    # ista osnova
```

Lematizacija — prava, preko biblioteke classla (model za srpski, procesori tokenize, pos, lemma). Za razliku od stemera, vraća rečničku lemu i hvata nepravilne oblike (najbolji→dobar). Lazy-inicijalizacija i keširanje modela, batch obrada radi brzine.

Bez curenja informacija: pošto su stem/lemma deterministički po tokenu, pretprocesiranje se radi *jednom* nad celim skupom; vokabular i IDF se uče tek unutar svakog fold-a (vidi sledeću sekciju).

Teorija — zašto uopšte normalizovati reči

Klasični modeli vide tekst kao **vreću reči** (engl. *bag of words*): svaka različita reč je zasebno obeležje (kolona). Srpski je *morfološki bogat* — ista reč ima desetine oblika (pobeda, pobede, pobedom, pobedi...). Bez normalizacije model svaki oblik uči zasebno, pa se signal *raspršuje* po mnogo retkih kolona i modelu brzo ponestane primera po obeležju (problem *retkosti/sparsnosti*). Stemovanje i lema-

tizacija spajaju te oblike u jednu osnovu $\Rightarrow$ manje dimenzija, gušće kolone, bolja generalizacija. **Stem vs lema:** stem je brza heuristika (može pogrešiti), lema je morfološki tačna ali traži model — zato obe poredimo kao varijante. Lowercasing i transliteracija ćirilice su ista ideja: spajanje površinski različitih, a značenjski istih tokena.

5 Faza 3a — Osnovni modeli (src/baseline/)

5.1 run_baseline.py — grid eksperimenata

Poredi logističku regresiju i multinomijalni naivni Bajes preko scikit-learn, nad svim kombinacijama: normalizacija (none/stem/lemma) $\times$ ponderisanje (bow/tf/idf/tfidf) $\times$ n-gram (1-1 / 1-2).

Teorija — kako tekst postaje brojevi (vektORIZACIJA)

Model ne razume reči, samo brojeve. **Vektorizacija** pretvara naslov u vektor: napravi se *vokabular* svih reči u korpusu, i svaki naslov se predstavi redom brojeva — po jedan za svaku reč iz vokabulara (koliko se puta javlja). To je **vreća reči** (redosled se gubi, samo se broji prisustvo). Primer sa vokabularom {zvezda, pobedila, šok, transfer}:

Žvezda pobedila" $\rightarrow$ [1, 1, 0, 0] "Šok transfer" $\rightarrow$ [0, 0, 1, 1]

Pošto vokabular ima desetine hiljada reči, a naslov tek nekoliko, vektori su *ogromni i skoro svi nule* (retki/sparse). Zato su važni i normalizacija (da oblika bude manje) i min_df (da retke reči ne naduvaju vokabular).

Šeme ponderisanja se mapiraju na parametre sklearn vektorizatora:

```
bow    : CountVectorizer()                # puko brojanje
tf      : TfidfVectorizer(use_idf=False, norm="l2")    # term-frekvencija
idf     : TfidfVectorizer(binary=True, use_idf=True)   # prisustvo  $\times$  IDF
tfidf   : TfidfVectorizer(use_idf=True, sublinear_tf=True, norm="l2") # pun TF-IDF
# zajedničko: token_pattern=r"\S+" (tekst je već tokenizovan), min_df=2
```

Teorija — TF, IDF i TF-IDF ponderisanje

Pošto reči nisu jednako informativne, broj pojavljivanja se *pondériše*:

- **TF** (*term frequency*) — koliko se reč javlja u naslovu. Česta reč u dokumentu je važnija za taj dokument. (norm="l2" skalira vektor na jediničnu dužinu da duži naslovi ne dominiraju.)
- **IDF** (*inverse document frequency*) — $\log(N/df)$: reč koja se javlja u *svakom* naslovu (npr. „je”, „u”) nosi malo informacije i dobija malu težinu; retka, specifična reč dobija veliku.
- **TF-IDF** = TF $\times$ IDF — spaja oba: visoko vrednuje reč koja je *česta u ovom* naslovu, a *retka u korpusu* (tj. diskriminativna). sublinear_tf primeni $1 + \log(\text{TF})$ jer 10 pojava nije 10 $\times$ važnije od jedne.

Mali primer IDF-a: u korpusu od 2200 naslova reč „je” se javlja u ≈ 2000 , a „transfer” u 40. Onda $\text{IDF}(\text{je}) = \log(2200/2000) \approx 0,1$, a $\text{IDF}(\text{transfer}) = \log(2200/40) \approx 4,0$ — „transfer” dobija $\sim 40\times$ veću težinu jer je informativniji.

Zašto baš ove varijante poredimo: na *kratkim* naslovima reči se retko ponavljaju, pa $\text{TF} \approx \text{prisustvo}$ i IDF nosi glavnu razliku — empirijski proveravamo koliko to vredi. min_df=2 izbacuje reči iz samo jednog naslova (verovatno šum/tipfeler, i sprečavaju preprilagodavanje). **N-grami:** unigram (1-1) gleda reči pojedinačno; bigram (1-2) dodaje i parove susednih reči — npr. „velika” i „vest” zasebno su neutralne, ali bigram „velika vest” je signal. Zato 1-2 ume da uhvati fraze koje vreća pojedinačnih reči promaši.

Bitno: lowercase=False i token_pattern=\S+ jer je tekst već tokenizovan u serbian.py — vektorizator samo deli po razmaku, ne tokenizuje ponovo.

Pošteno ocenjivanje — ugnežđena unakrsna validacija. Da se izbegne curenje informacija, koristi se *nested CV*: spoljna 10-struka petlja meri performansu, a unutrašnja bira hiperpara-

metre (`GridSearchCV`). Vektorizator je deo `Pipeline`-a, pa se vokabular/IDF uče *samo* na trening delu svakog fold-a:

```
outer = StratifiedKFold(10, shuffle=True, random_state=42)
for tr, te in outer.split(X, y):
    pipe = Pipeline([("vec", make_vectorizer(...)), ("clf", est)])
    gs = GridSearchCV(pipe, grid, scoring="f1", cv=inner) # bira C / alpha
    gs.fit(X[tr], y[tr]) # uči SAMO na trening foldu
    y_pred = gs.best_estimator_.predict(X[te])
```

Teorija — zašto baš *ugneždjena* i *stratifikovana* CV

Problem koji rešava: ako bismo testirali na jednom fiksnom delu podataka, rezultat bi zavisio od sreće u tom razdvajanju. **Unakrsna validacija (CV)** deli skup na k jednakih delova (fold-ova); k puta trenira na $k-1$ dela i testira na preostalom (svaki deo jednom bude test), pa usrednji — tako se meri na *svim* podacima. Mi koristimo $k=10$ (kompromis: dovoljno trening podataka po fold-u, a ocena stabilna). **Stratifikovana** = svaki fold čuva odnos klasa 50/50; inače bi neki fold mogao slučajno dobiti npr. 70 % klikbejta i iskriviti metriku. **Zašto *ugneždjena* (nested):** hiperparametre treba izabrati *ne gledajući* podatke na kojima prijavljujemo rezultat. Zato dve petlje: *unutrašnja* bira C/alpha samo unutar trening dela, a *spoljna* ocenjuje na fold-u koji pri tom izboru nije korišćen. Da koristimo jednu petlju za oboje, „provirili” bismo u test $\Rightarrow$ lažno bolji (optimistično pristrasan) rezultat. `random_state=42` čini podele ponovljivim (svako dobije iste fold-ove).

Hiperparametri — čemu služe i zašto ti opsezi. Hiperparametri se ne uče iz podataka nego se zadaju i biraju validacijom. Mi tražimo po log-skali (geometrijski korak), jer njihov efekat je multiplikativan, a ne linearan:

```
LR (logistička regresija): C in {0.01, 0.1, 1, 10, 100}
MNB (naivni Bajes): alpha in {0.01, 0.1, 0.5, 1, 2}
```

Teorija — C (regularizacija) i alpha (izgladivanje)

C kod logističke regresije kontroliše *regularizaciju* — kaznu za prevelike težine. Model minimizuje (greška) $+\frac{1}{C}\|w\|$. *Malo C* $\Rightarrow$ jaka kazna $\Rightarrow$ jednostavniji model, manje preprilagođavanja (*overfitting*), ali rizik od *podprilagođavanja*. *Veliko C* $\Rightarrow$ slaba kazna $\Rightarrow$ model se pripije uz trening podatke (rizik *overfittinga*). Pošto se prava vrednost ne zna unapred, probamo opseg 0,01–100 i pustimo validaciju da izabere. Kod kratkih, retkih tekstova regularizacija je važna jer obeležja ima mnogo, a primera malo.

alpha kod naivnog Bajesa je *Laplace/aditivno izgladivanje*. Naivni Bajes množi verovatnoće reči; ako se neka reč u trening skupu nikad nije pojavila uz klasu, njena verovatnoća je 0 i *poništi* ceo proizvod. **alpha** dodaje malu „pseudo-frekvenciju” svakoj reči da nijedna ne bude nula. *Veće alpha* = glađe/opreznije procene (više se oslanja na prior), *manje alpha* = vernije trening podacima. Opet biramo validacijom u 0,01–2.

Zašto log-skala: skok sa $C=1$ na 2 je beznačajan, ali sa 1 na 100 menja ponašanje modela — zato koraci idu $\times 10$, a ne $+1$ (efikasnije pokriva prostor sa malo tačaka).

Kako prepoznati pravu vrednost — over/underfitting. *Overfitting* (preveliko C): model je odličan na trening podacima, ali slab na test — „napamet” je naučio, ne generalizuje. *Underfitting* (premalo C): slab i na trening i na test — premoćna kazna ga je učinila pretupavim. Idealni hiperparametar je u sredini, gde je test rezultat najveći — a upravo to bira unutrašnja validacija.

Teorija — dva modela: zašto upravo NB i LR

Multinomijalni naivni Bajes je verovatnosni klasifikator: prepostavlja da su reči *nezavisne* u datoj klasi (otud „naivni”) i računa koja klasa čini naslov verovatnijim. Brz, jak na malom tekstu, klasičan baseline za klasifikaciju teksta. **Logistička regresija** je linearni model koji uči težinu svake reči i kombinuje ih u verovatnoću klikbejta; ne prepostavlja nezavisnost i daje dobro kalibrisane verovatnoće. Dva komplementarna pristupa daju pošteniju „donju granicu” koju transformeri treba da nadmaše.

5.2 common.py — učitavanje i metrike

`load_dataset` čita `dataset.tsv`; `compute_metrics` računa iste metrike za sve modele (radi poredjenja): tačnost, preciznost/odziv/**F1 za klikbejt klasu**, makro-F1 i **ROC-AUC** (kada model daje verovatnoće).

Teorija — preciznost, odziv, F1, ROC-AUC

Svaka predikcija pada u jedno od četiri polja *matrice konfuzije*: **TP** (tačno pogodan klikbejt), **FP** (regularan pogrešno proglašen klikbejtom, „lažna uzbuna”), **FN** (klikbejt promašen), **TN** (tačno pogodan regularan). Iz njih:

$$\text{preciznost} = \frac{TP}{TP + FP}, \quad \text{odziv} = \frac{TP}{TP + FN}.$$

Preciznost = od svih koje je model nazvao klikbejtom, koliko ih je stvarno to. **Odziv** (*recall*) = od svih stvarnih klikbejtova, koliko ih je uhvatio. Njih dvoje su u tenziji: ako model agresivno svuda viče „klikbejt”, odziv raste ali preciznost pada (mnogo FP). **Primer:** od 100 stvarnih klikbejtova model uhvati 70 ($TP=70, FN=30 \Rightarrow$ odziv 0,70), ali uz to 30 regularnih proglasi klikbejtom ($FP=30 \Rightarrow$ preciznost $70/100 = 0,70$). **F1** je harmonijska sredina preciznosti i odziva — jedan broj koji je visok samo ako su *oba* visoka (kažnjava disbalans jače od obične sredine). **Zašto F1 baš na klikbejt klasi:** nju želimo da detektujemo; sama tačnost bi bila varljiva (model koji sve proglasi regularnim ima 50 % tačnosti na balansiranom skupu, a nula korisnosti). **Makro-F1** usrednji F1 obe klase ravnopravno. **ROC-AUC** meri koliko dobro model *rangira* naslove po verovatnoći, nezavisno od praga 0,5: 1,0 savršeno, 0,5 nasumično — imaju ga samo modeli koji daju verovatnoću (NB, LR, BERT), a ne dekođer sa tvrdom 0/1 odlukom.

Izlaz: `results/baseline_results.csv`. Najbolje: *naivni Bajes + stem + TF-IDF (1-2)*, $F1 \approx 0,646$.

6 Faza 3b — Enkoderski LLM (src/transformers/)

6.1 finetune.py — fine-tuning BERTić i mBERT

Fino podešava dva enkoderska transformera preko HuggingFace Trainer-a: **BERTić** (`classla/bcms-bertic` monolingvalni za BCMS) i **mBERT** (`bert-base-multilingual-cased`, višezječni). Protokol: 10-struka CV, variranje broja epoha (2/3/4). Traži GPU — pokreće se na Colab-u.

Teorija — šta je BERT, enkoder vs dekođer, fine-tuning

Transformer je neuronska mreža koja čita ceo tekst odjednom i preko mehanizma *pažnje* (*attention*) za svaku reč gleda *kontekst* (ostale reči). Intuicija pažnje: pri obradi reči „šok”, model „obrat pažnju” na okolne reči da proceni da li je u pitanju senzacionalizam („šok u Humskoj”) ili stvarni sadržaj („šok terapija posle povrede”) — ista reč, različito značenje. Otud „kontekstualni” modeli: ista reč dobija različitu reprezentaciju zavisno od okoline, za razliku od vreće reči koja je slepa za kontekst (zato baseline teško hvata suptilni klikbejt). **Enkoder** (BERT) je predtreniran da *razume* tekst — uči tako što se zamaskira reč i model je pogađa gledajući kontekst *i levo i desno*; idealan za klasifikaciju. **Dekođer** (GPT, Claude) je predtreniran da *generiše* — predviđa sledeću reč gledajući samo unazad; koristimo ga u Fazi 3c. **Fine-tuning** = uzmemo model već *predtreniran* na milijardama reči (opšte znanje jezika) i kratko ga doučimo na našem malom označenom skupu, da to znanje usmerimo baš na klikbejt — mnogo jeftinije i bolje nego učiti od nule. **Mono vs multi:** BERTić je treniran samo na srpskom i srodnim jezicima, pa sav kapacitet troši na njih; mBERT isti kapacitet deli na 100+ jezika — zato BERTić bolje hvata nijanse i pobeđuje.

Tehnički zanimljiv detalj — sudar imena. Folder se zove `transformers`, isto kao HuggingFace biblioteka. Da `import transformers` ne učitava *lokalni* folder umesto biblioteke, kod na startu čisti `sys.path` i keš modula:

```
sys.path[:] = [p for p in sys.path if p not in ("", _SELF, _SRC)]
sys.path.append(_SRC)    # 'src' ide na KRAJ (da se 'baseline' i dalje nađe)
# + izbaci eventualno već uvezen lokalni 'transformers' iz sys.modules
```


Tokenizacija za transformer je drugačija od baseline-a: koristi se *subword* tokenizator samog modela (AutoTokenizer), sa odsecanjem na `max_len`. Za svaki fold pravi se *svež* model i trenira tačno *n* epoha:

```
tokenizer = AutoTokenizer.from_pretrained(hf_id)
model = AutoModelForSequenceClassification.from_pretrained(hf_id, num_labels=2)
```

Teorija — subword tokenizacija i hiperparametri obuke

Subword tokenizacija deli reč na česte pod-delove (npr. `pobedom` → `pobed+##om`). Time model nema „nepoznatih reči”: svaku, i retku, sastavi od delova — posebno korisno za morfološki bogat srpski. Zato ovde *ne* radimo stemovanje: model sam uči podreči. Ključni hiperparametri obuke:

- **Broj epoha** (2/3/4) — koliko puta model prođe ceo trening skup. Premalo ⇒ ne nauči (podprilagodavanje); previše ⇒ zapamti trening (overfitting). Zato *variramo* i biramo: 3–4 je optimum (kriva učenja se izravna).
- **max_len** — maksimalna dužina u tokenima; naslovi su kratki pa se gotovo ništa ne odseca, a računanje je brže.
- **batch_size** — koliko primera ide kroz model pre ažuriranja težina; veći batch = stabilnije, ali traži više GPU memorije.
- **learning_rate** — veličina koraka pri učenju; za fine-tuning je mali (npr. $2 \cdot 10^{-5}$) da se ne „pokvari” predtrenirano znanje.

Zašto svež model po fold-u: da svaki fold bude nezavisan eksperiment, bez prenosa naučenog iz prethodnog (inače curenje informacija).

ROC-AUC se dobija *softmax*-om nad logitima (verovatnoća klikbejt klase):

```
proba = softmax(logits)[: , 1] # za roc_auc_score
```

Teorija — logiti i softmax

Model na izlazu daje *logite* — sirove brojeve za svaku klasu, koji nisu verovatnoće. **Softmax** ih pretvara u verovatnoće (pozitivne, sabiraju se u 1). Uzimamo verovatnoću klase 1 (klikbejt) da bismo mogli da računamo ROC-AUC, koji traži rangiranje po verovatnoći, a ne samo tvrdu odluku.

Izlaz: `results/encoder_bertic_results.csv`, `encoder_mbert_results.csv`. Najbolje: BERTiĆ, 3–4 epohe (F1≈0,703, ROC-AUC≈0,788). `colab_train.py` je verzija prilagođena Colab okruženju.

7 Faza 3c — Dekoderski LLM (src/decoder/)

7.1 prompts.py — konstrukcija upita

Definiše prompt varijante koje se porede: jezik (SR/EN) × format (zero/few-shot) × sa/bez definicije klikbejta. `build_prompt` sklapa (`system`, `user_template`); ključno je da se od modela traži **samo cifra**:

```
rule = "Odgovori ISKLJUČIVO jednom cifrom: 1 (klikbejt) ili 0 (regularan)."
```

```
user = f'{task}\n\n{definicija}{examples}Naslov: "{naslov}"\n\nOdgovor: '
```

Few-shot varijante ubacuju 6 rešenih primera (FEWSHOT); ti naslovi se po potrebi izuzimaju iz evaluacije (holdout) da ne bi „procurili”.

Teorija — zero-shot vs few-shot, uloga definicije

Zero-shot = modelu damo samo zadatak, bez ijednog rešenog primera — oslanja se isključivo na znanje iz predtreniranja. **Few-shot** = u prompt ubacimo nekoliko rešenih primera (ovde 6) iz kojih model „na licu mesta” nauči obrazac (*in-context learning*), bez menjanja težina. Zato few-shot primeri *ne* smeju biti u test skupu — inače merimo memorisanje, ne generalizaciju (otud *holdout*). **Definicija klikbejta**

u promptu je treći faktor: usmerava model na *naše* shvatanje pojma. Sve tri ose (jezik, broj primera, definicija) variramo da vidimo šta najviše pomaže — to je „prompt inženjering” kao zamena za obuku.

7.2 `claude_eval.py` — evaluacija nad celim skupom

Šalje svaki naslov Anthropic Claude-u (Haiku), zero-shot, i parsira 0/1 odgovor. `max_tokens=16` jer je odgovor jedna cifra (jeftino i brzo):

```
resp = client.messages.create(model=model, max_tokens=16,
                              system=system, messages=[{"role": "user", "content": user_prompt}])
```

Teorija — zašto `max_tokens=16` i bez *temperature*

`max_tokens` ograničava dužinu odgovora; pošto tražimo samo cifru 0/1, 16 je sasvim dovoljno i *jeftinije* (plaća se po tokenu). *Temperature* kontroliše nasumičnost generisanja — za klasifikaciju želimo *determinističan*, ponovljiv odgovor, pa je ne dižemo (na nekim modelima bi je slanje čak odbilo). Sistemski prompt izričito traži samo cifru da bi parsiranje (`parse_label`) bilo pouzdano.

Tri inženjerska detalja koja čine evaluaciju pouzdanom i jeftinom:

1. **Keširanje** (`results/decoder_cache/*.jsonl`) — svaki rezultat se upiše; ponovno pokretanje preskače već klasifikovane naslove (prekid/nastavak bez dvostrukog plaćanja).
2. **Retry sa eksponencijalnim odlaganjem** — na privremenu grešku/rate limit pokušava do 5 puta (`2**attempt`).
3. **Ograničavanje učestalosti** (`min_interval`) — razmak između poziva, da se ne probije API limit.

Izlaz: `results/decoder_results.csv` + log. Najviši F1 na klikbejt klasi u celom radu ($\approx 0,715$), ali samo tvrde 0/1 odluke (bez ROC-AUC). `gemini_eval.py` i `chatgpt_eval.py` su alternativni dekoderi (dele keš i parsiranje).

8 Faza 4 — Izveštaj (`src/report/` + `report/`)

8.1 `make_tables.py` — CSV → LaTeX tabele

Čita `results/*.csv` i piše booktabs tabele spremne za `\input` u izveštaj (`report/tables/`). Bira najbolju varijantu po tipu modela za zbirnu tabelu poređenja. Otporno na nedostajuće ulaze — ako CSV ne postoji, upiše placeholder da se izveštaj svejedno kompajlira.

8.2 `make_figures.py` — grafici

`matplotlib` grafici: raspodela klasa (bar), histogram dužine naslova po klasi, kriva učenja (F1/AUC po epohi) i poređenje modela. Svaki se snima kao PNG (150 dpi) u `report/figures/`.

8.3 `izvestaj.tex` — glavni dokument

LaTeX izveštaj (`pdfLaTeX` / `tectonic`); `\input`-uje generisane tabele i uključuje grafike. Kompajlira se u `report/izvestaj.pdf`. Bibliografija je u `references.bib`.

9 Kako sve teče zajedno (reprodukcija)

Ceo lanac, ukratko (pune komande sa očekivanim izlazom su u `docs/Komande.md`):

1. `sportklub_api.py` → sirovi naslovi.
2. `make_splits.py` → (ručna anotacija) → `iaa.py` (kappa) → `build_dataset.py` → `dataset.tsv`.
3. `run_baseline.py`, `finetune.py`, `claude_eval.py` → `results/*.csv`.

4. `make_tables.py + make_figures.py` $\rightarrow$ `tectonic izvestaj.tex` $\rightarrow$ **PDF**.

Prateći dokumenti: `docs/Mapa-koda.md` (kratka mapa), `docs/Obrada-teksta.md` (tokenizacija-stemovanje/lematizacija detaljno), `docs/Komande.md` (komande + očekivani izlaz).